

Tecnología Digital VI

Árboles de decisión

Licenciatura en Tecnología Digital - UTDT

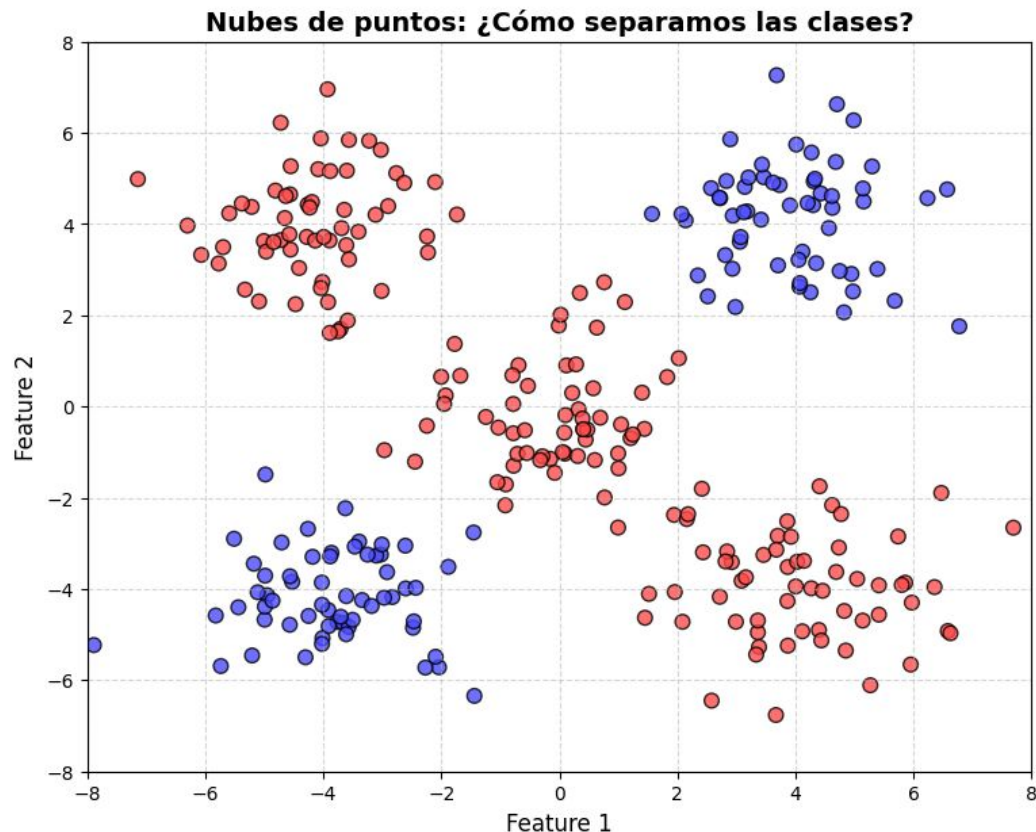
Árboles de decisión

Hasta ahora tenemos los **modelos lineales** para hacer **predicciones**. Si bien tienen algunas propiedades interesantes (interpretación y explicabilidad), **son bastante restrictivos para la mayoría de los problemas reales** (quiero EL mejor hiperplano). El poder de cómputo que tenemos hace ya muchos años nos permite buscar más opciones, más sectorizadas para diferentes partes del dataset, con diferentes pesos en las variables, etc.

Vamos a continuar nuestro camino de modelos por los Árboles de decisión.

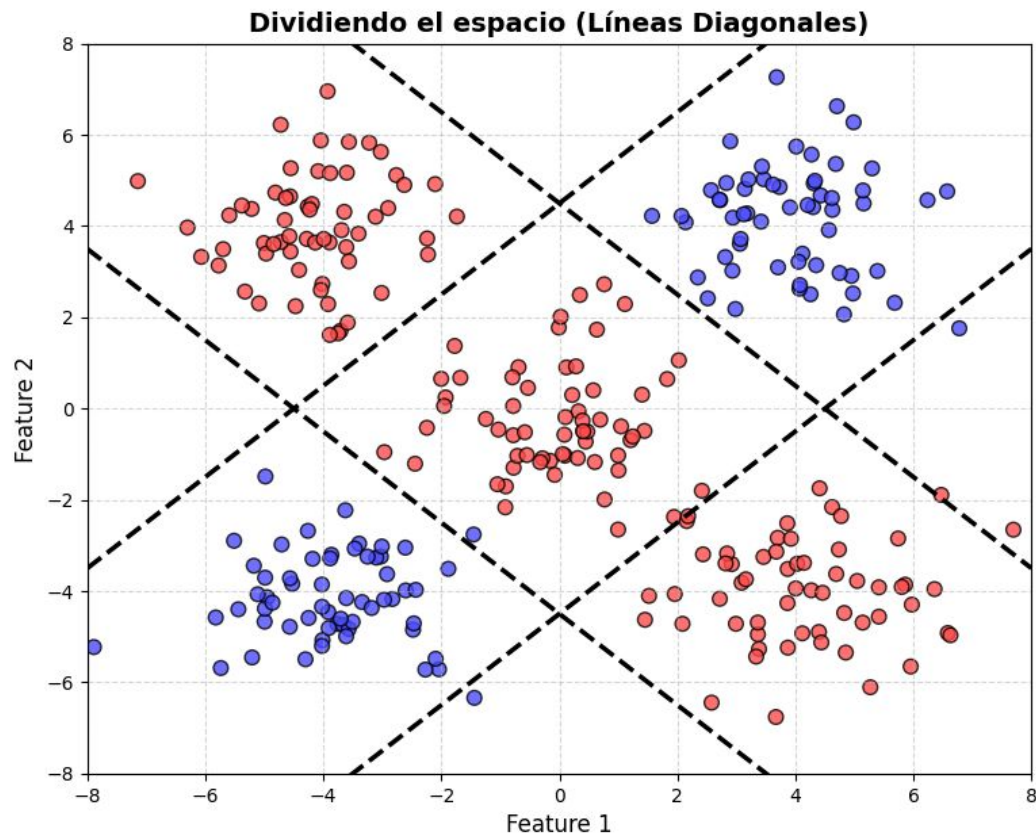
Intuición en clasificación

Supongamos que ya no tenemos una sola línea como en regresión logística, ahora nos permiten dibujar muchas líneas rectas. ¿Cómo dividimos este dataset?



Intuición en clasificación

Esta es una opción. Tal vez hay más simples y con menos líneas, pero no estábamos limitados por el momento.



Árboles de decisión

¿Qué hace entonces un árbol de decisión en un problema de clasificación? Parte el espacio de atributos en diferentes regiones. Cada región queda asignada a una clase posible. La partición se hace tratando de minimizar el error de categorización, es decir que de alguna manera los puntos caigan en la clase que tienen que caer.

Vamos a tener diferencias con el proceso que acabamos de hacer intuitivamente:

- Lo “malo”: No podemos usar diagonales, solo rectas paralelas a los ejes.
 - Lo “bueno”: Tenemos la posibilidad de solo partir regiones ya partidas.
- Es decir, las rectas puede ser semirrectas o segmentos.

Se pueden dividir las regiones que ya fueron divididas

Formalización

Vimos la intuición en un problema de clasificación. Vamos a formalizar lo que hace el método en un problema de regresión (y luego volveremos a clasificación).

Las particiones que buscábamos se hacían tratando de que los puntos “caigan en donde tienen que caer”. ¿Y si no pueden caer todos en el lugar correcto? Si tengo que dejar a algunos afuera, ¿en qué fórmula me baso para saber cuál?

Lo que estamos buscando es una loss function. La métrica de optimización del algoritmo por la cual va a saber qué cortes buscar.

Loss function

En una regresión, una métrica candidata va a ser el MSE, dado que después también va a ser útil como métrica de evaluación.

Si comienzo mi proceso con una gran única región (R^2 en el caso de los gráficos que estuvimos viendo), entonces lo mejor que puedo hacer para minimizar el MSE con una única predicción es predecir para todos el promedio de las muestras.

De esta manera, el MSE para una región m será:

$$MSE_m = \frac{1}{N_m} \sum_{i \in m} (y_i - \hat{y}_m)^2$$

Una región para todos no puede explicar mucho, comencemos a dividir el espacio. Por el momento supongamos que agarramos algún feature cualquiera de los disponibles, y ponemos el corte por algún valor cualquiera del feature (los menores e iguales “caen de un lado”, los mayores del otro).

Si simplemente parto el espacio, pero no actualizo la predicción en absoluta, la loss function continuará igual:

$$MSE_m = \frac{1}{N_m} \left[\sum_{i \in m_1} (y_i - \hat{y}_m)^2 + \sum_{i \in m_2} (y_i - \hat{y}_m)^2 \right]$$

Se calcula el ECM de la nueva región creada

Actualización de la predicción ^

Split

La idea es que ahora cada región se tiene que encargarse de explicarse a sí misma. Para los puntos de m_1 vamos a predecir su propio promedio, y lo análogo haremos con m_2 .

Ahora la loss function nos queda:

$$MSE_{split} = \frac{1}{N_m} \left[\sum_{i \in m_1} (y_i - \hat{y}_{m_1})^2 + \sum_{i \in m_2} (y_i - \hat{y}_{m_2})^2 \right]$$

Totalidad del MSE de todos los splits

Región m1

Región m2

Con unos pequeños cambios, podemos ver que la fórmula vista es equivalente a la suma ponderada de los MSE de cada región:

$$MSE_{split} = \frac{N_{m_1}}{N_m} MSE_{m_1} + \frac{N_{m_2}}{N_m} MSE_{m_2}$$

Ejemplo

Hagamos un ejemplo muy simple a mano.

Tenemos 5 alumnos que en el examen se sacaron: 2, 4, 6, 8, 10. Sabemos que estudiaron en cantidad de horas, respectivamente: 1, 2, 3, 8, 9.

Queremos predecir la nota en función de las horas de estudio con un árbol de decisión de *profundidad 2*.

¿Cuál sería el mejor split posible? ¿Cómo quedaría el MSE?

x_1 = # horas, y = notas
las filas representan a cada alumno

El MSE no puede empeorar si se hacen más splits

Encontrando el split

Ya podemos definir cómo queda la loss function al hacer un split, y pudimos encontrar un split *a mano*. ¿Cómo hace el algoritmo de optimización para encontrar el mejor split de manera metódica?

Sin complicarse, fuerza bruta.

- Va a probar con cada feature posible.
- Por cada feature: va a probar con todos los valores posibles. -> VISTOS en el dataset
- Entre todas estas opciones, se queda con la que minimice MSE_{split} .

Al árbol no le importa si las variables están escaladas o no, sólo si una clasificación cae de algún lado o del otro

Es claro qué significa *cada feature posible* pero, ¿qué significa *todos los valores posibles*?

Si el feature es un valor discreto (como fue la cantidad de horas), puedo probar todos. ¿Qué hago si es un valor continuo?

Subdividiendo

Ahora tenemos un algoritmo bien definido para encontrar el mejor split, pero cambiar de una gran región a dos regiones no va a representar una mejora determinante.

¿Qué hacemos a continuación? Subdividimos recursivamente cada subregión presente.

¿Hasta cuando? Hasta cuando queramos, pero con cuidado del overfitting.

Clasificación

Ahora que tenemos el algoritmo completo con su métrica para regresión, volvamos a **clasificación**. El algoritmo es el mismo, pero la métrica para ver qué tan bien estamos clasificando *tiene* que cambiar.

¿Cuál podemos usar?

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$

$$H_m = - \sum_{k=1}^C p_{mk} \log_2(p_{mk})$$

Clasificación

Ambas son medidas de *impureza*. Miden qué tan *impuro* es el nodo en el sentido de qué tan mezclada están las muestra al mirar su categoría.

Cuando todos los nodos son de la misma clase, las medidas de impureza dan 0. Cuando todos los nodos son de diferentes clases, las medidas de impureza llegan a su máximo.

El coeficiente de desigualdad de Gini es de los más utilizados, aunque hay otros:

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$

Loss function

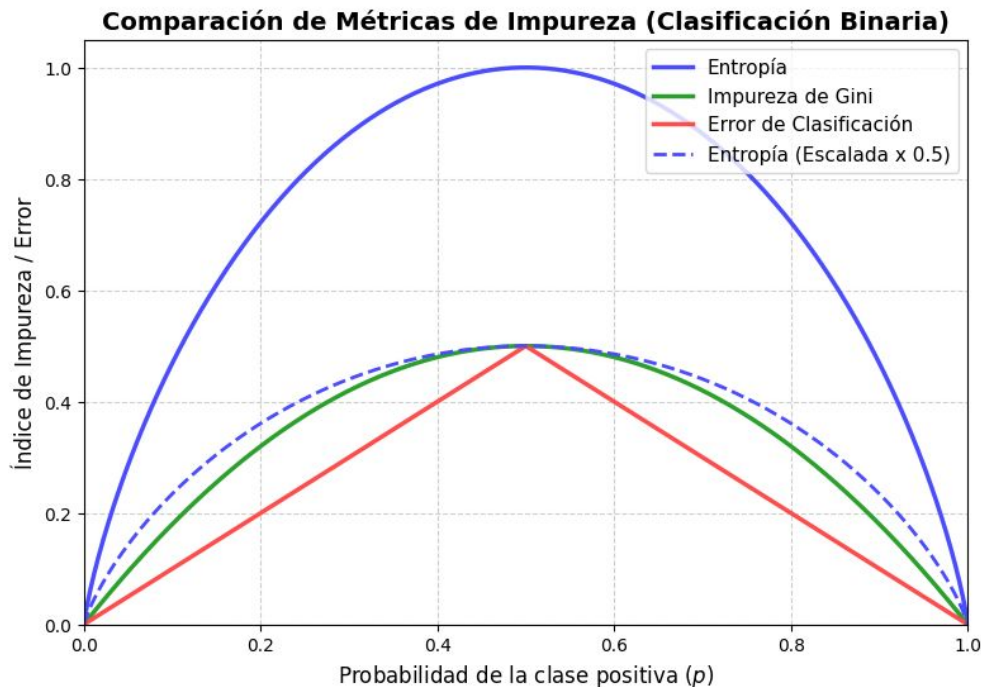
Una vez que tenemos una medida de *impureza* de un nodo, la función de costo a minimizar al hacer un split es simplemente ponderar las impurezas de cada subregión.

$$J(k, t_k) = \frac{N_{izq}}{N_m} I_{izq} + \frac{N_{der}}{N_m} I_{der}$$

Donde k es el feature y t_k el umbral. I es la medida de impureza elegida.

Optimización vs Evaluación

¿Por qué no usamos métricas que se parezcan más a la de evaluación?



Las medidas de impureza estiman mejor mientras más te acercas a los bordes

Regularización

Como todo método de Machine Learning, se necesitan mecanismos para no sobreajustarse a los datos de entrenamiento. En el caso de los árboles, si permitimos una flexibilidad excesiva, llegaremos a tener una región por cada muestra.

Algunos hiperparámetros posibles para manejar la flexibilidad:

- Profundidad Máxima del árbol (max_depth): Limitar la altura del árbol.
- Cantidad de muestras para dividir (min_samples_split): No partimos el nodo si no tiene más de n muestras.
- Cantidad de muestras en una hoja (min_samples_leaf): Cantidad mínima de muestras que debe tener una hoja.

Interpretabilidad

A diferencia de los modelos lineales, en los árboles tenemos un montón de decisiones en paralelo y en serie, en vez un solo coeficiente acompañando a cada feature. Interpretar nuestro modelo podría ser más complicado. Sin embargo, vamos a ver que podemos hacer varias cuestiones para interpretar nuestro modelo.

En primer lugar, podríamos directamente dibujar el árbol entero con todas sus decisiones (más cuando veamos ensambles). Por otro lado, podemos tener una medida de qué tan importante es cada una de las columnas del dataset mediante el gráfico de Feature Importance.

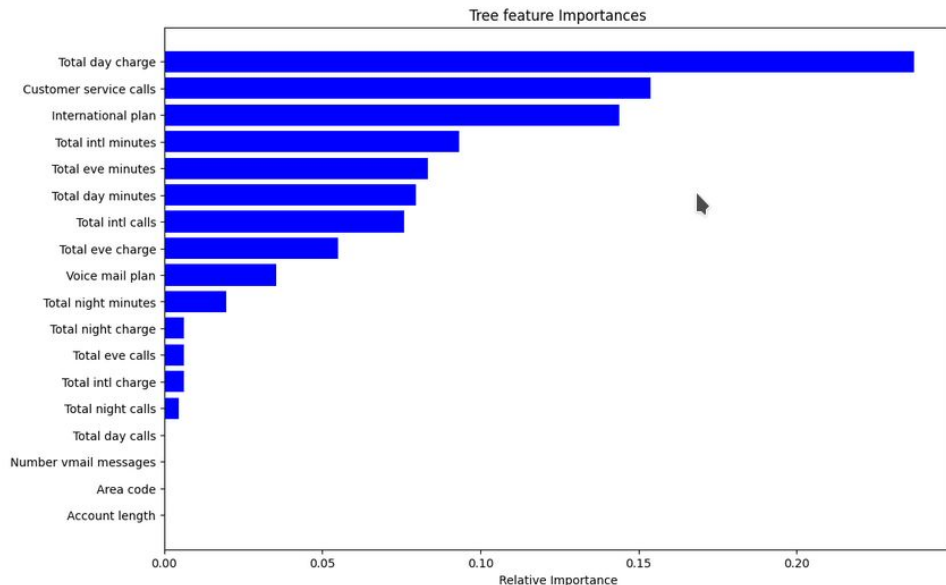
O sea digamos que un árbol es muchísimo más fácil de interpretar

Feature Importance

Un gráfico de Feature Importance nos ordena la importancia de cada columna en nuestro modelo.

Cómo se calcula:

- Por cada split del árbol, miramos la ganancia en la impureza y qué feature fue el responsable.
- Sumamos todas las ganancias aportadas y luego normalizamos a 1.



NaNs y categóricas

El algoritmo visto es directo si todas nuestras variables tienen valores numéricos y no hay valores faltantes.

¿Qué pasa si tenemos valores categóricos o NaNs? ¿Es necesario siempre preprocesar?

- Variables categóricas: Se pueden realizar cortes dejando ciertas categorías en cada lado. También se pueden realizar cortes múltiples aunque no es común.
- Valores faltantes: al momento de calcular el mejor split, se considera la opción de que todos los valores faltantes caigan de un lado o del otro, y se toma la mejor decisión.

Ventajas y Desventajas

Tenemos un nuevo modelo de Machine Learning ¿Qué nos aporta y a qué costo?

Ventajas:

- Suelen tener mejor rendimiento que los modelos lineales.
- Son simples de interpretar y visualizar.
- Pueden manejar valores categóricos y faltantes.
- Suele asemejarse a cómo los stakeholders toman decisiones.

Desventajas:

- Requieren más cómputo que los modelos lineales (aunque en general no es un cómputo excesivo).
- Vamos a ver modelos que van a tener casi siempre mejor performance.
- Pueden ser poco robustos y presentar grandes cambios en su estructura con pequeños cambios en el dataset.